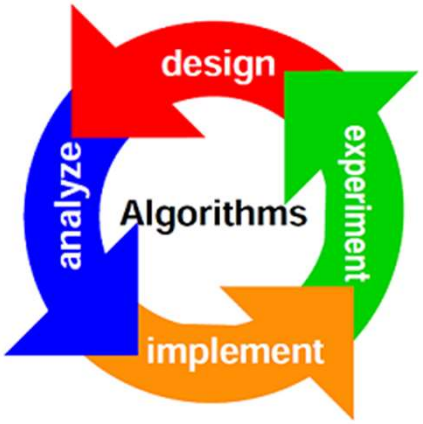


Algorithm analysis

Subject: Introduction to Algorithm

Lecturer: Hoang Do Thanh Tung

Credit points	3 ECTS
Level	Undergraduate
Teaching time	2024
Location	University of Science and Technology of Hanoi

- Reasons to analyze algorithms?
  - Correctness
  - Running time
- How to estimate the running time
  - Mathematical Background
  - What to analyze?
  - How to analyse?
  - Exercises

Reasons to analyze algorithms?

- Correctness
- Running time

Write down this algorithm

Input: b, n

Output: b^n

Process: $b^n = \underbrace{b \times \cdots \times b}_{n \text{ times}}.$

Your algorithm….



Smart power algorithm

- Idea:
  - $b^{1000} = b^{500} \times b^{500}$. If knowing b^{500}, we can compute b^{1000} with only 1 multiplication!
  - $b^{1001} = b^{500} \times b^{500} \times b$. If knowing b^{500}, we can compute b^{1001} with only 2 multiplication!
- **Smart power algorithm**:

  To compute b^n:

  Set p to 1, set h to b, and set m to n.
  Do While $m > 0$
  If m is an odd number then
  p= p.h
  m=m-1
  Else
  $m = m/2$
  h = h . h

  Print p

Analysis

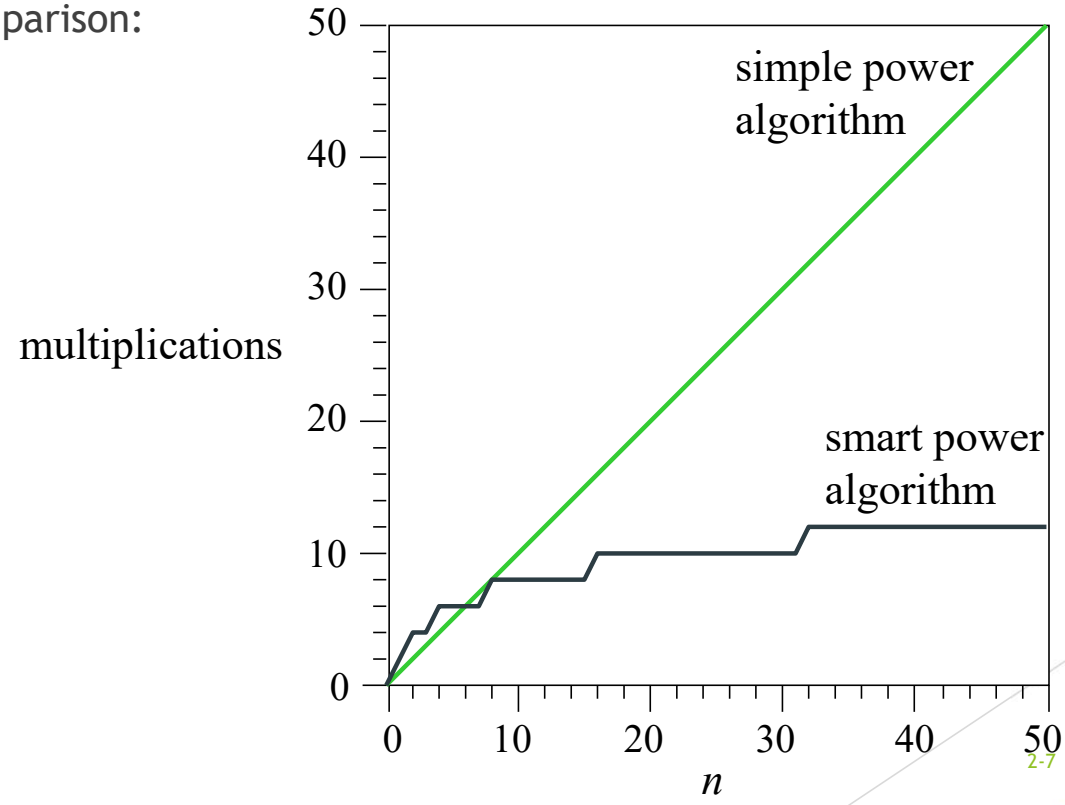
Analysis by counting multiplications

- the algorithm performs at most 2 multiplications before it halves the value of *n*. *It* are repeated until n reaches 0
- Assume c is number of multiplications, we know that $2^c=n$ so

$c = \log_2 n + 1$

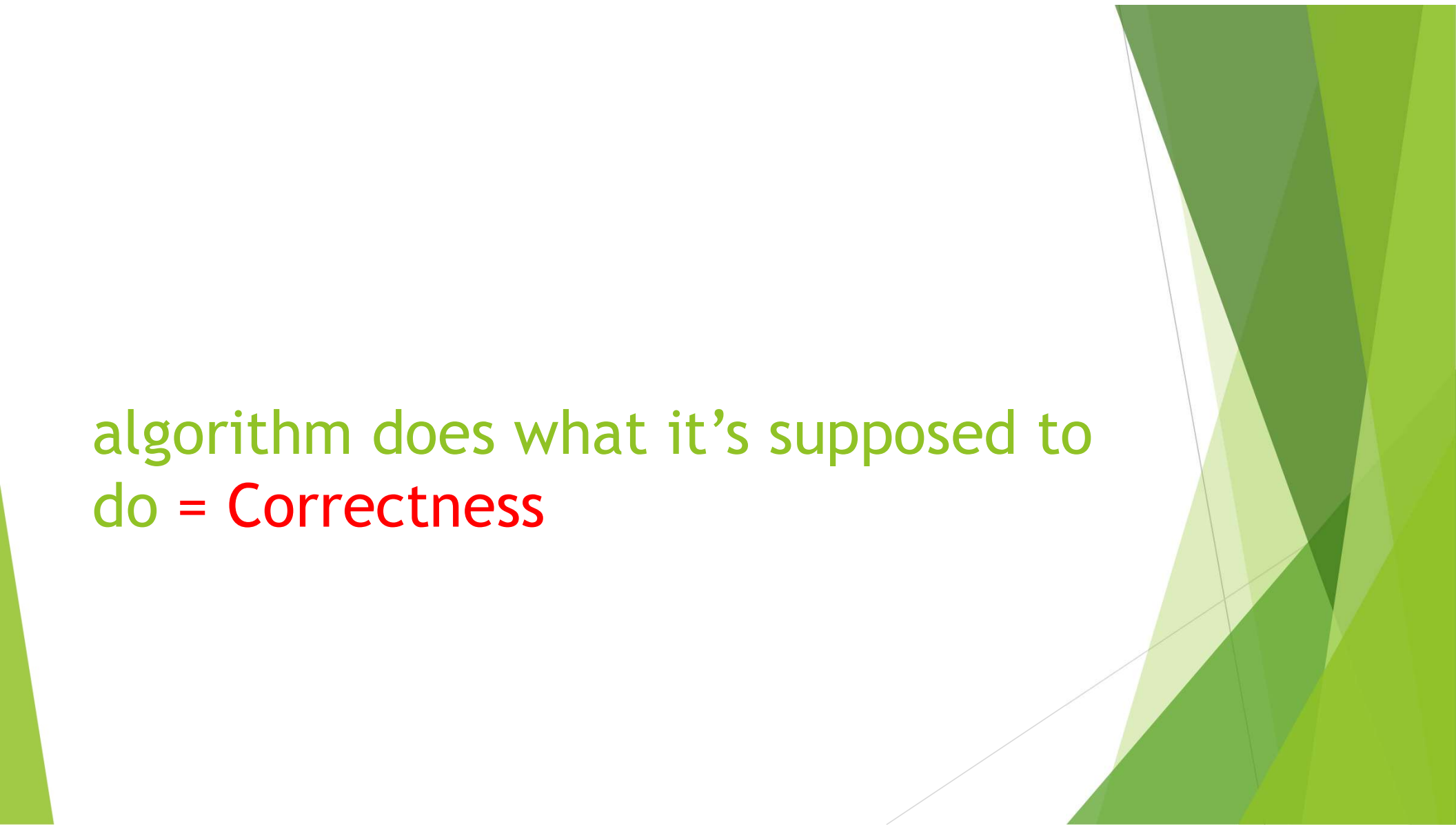
Compare two algorithms

- Comparison:



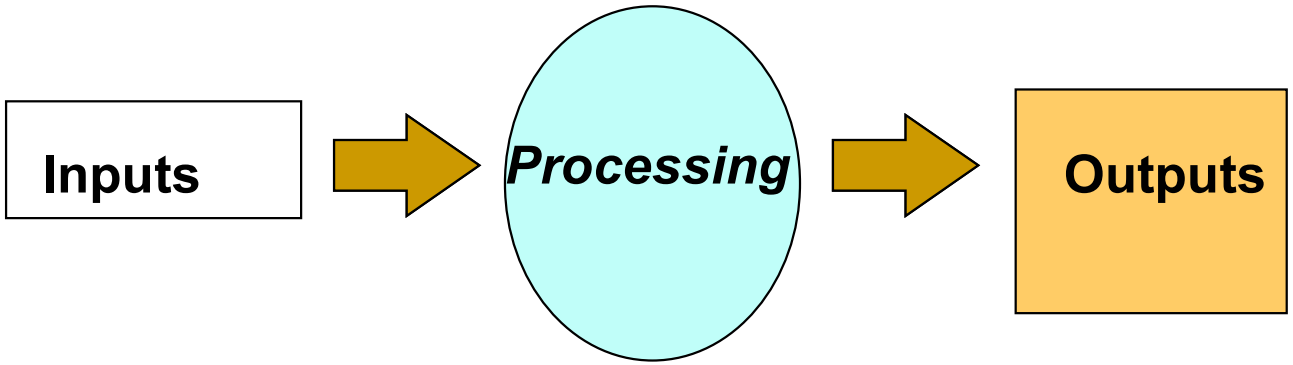

After writing down an algorithm,
we also need to convince ourselves that
• the algorithm does what it's supposed to do, and
• it does efficiently.

algorithm does what it’s supposed to do = Correctness



Is an algorithm Correct?

- We must prove that
  - Outputs are correct for all possible inputs
- *Trusting our instincts, or trying a few test cases, isn't good enough, even fairly obvious*
  - *'obvious' is all too often a synonym for 'wrong'*



input: A sequence of numbers.
output: An ordered permutation of the input.
issues: correctness, efficiency, storage, etc.

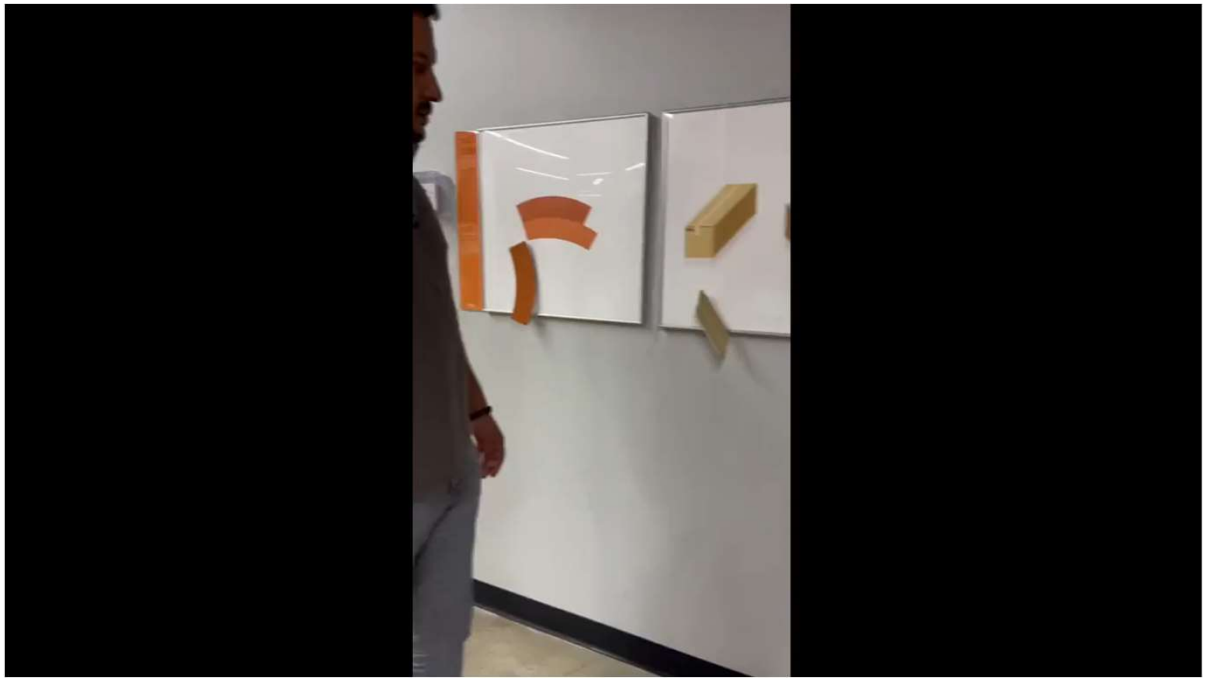
Input - Output



the obvious is obvious wrong



Don’t fool yourself

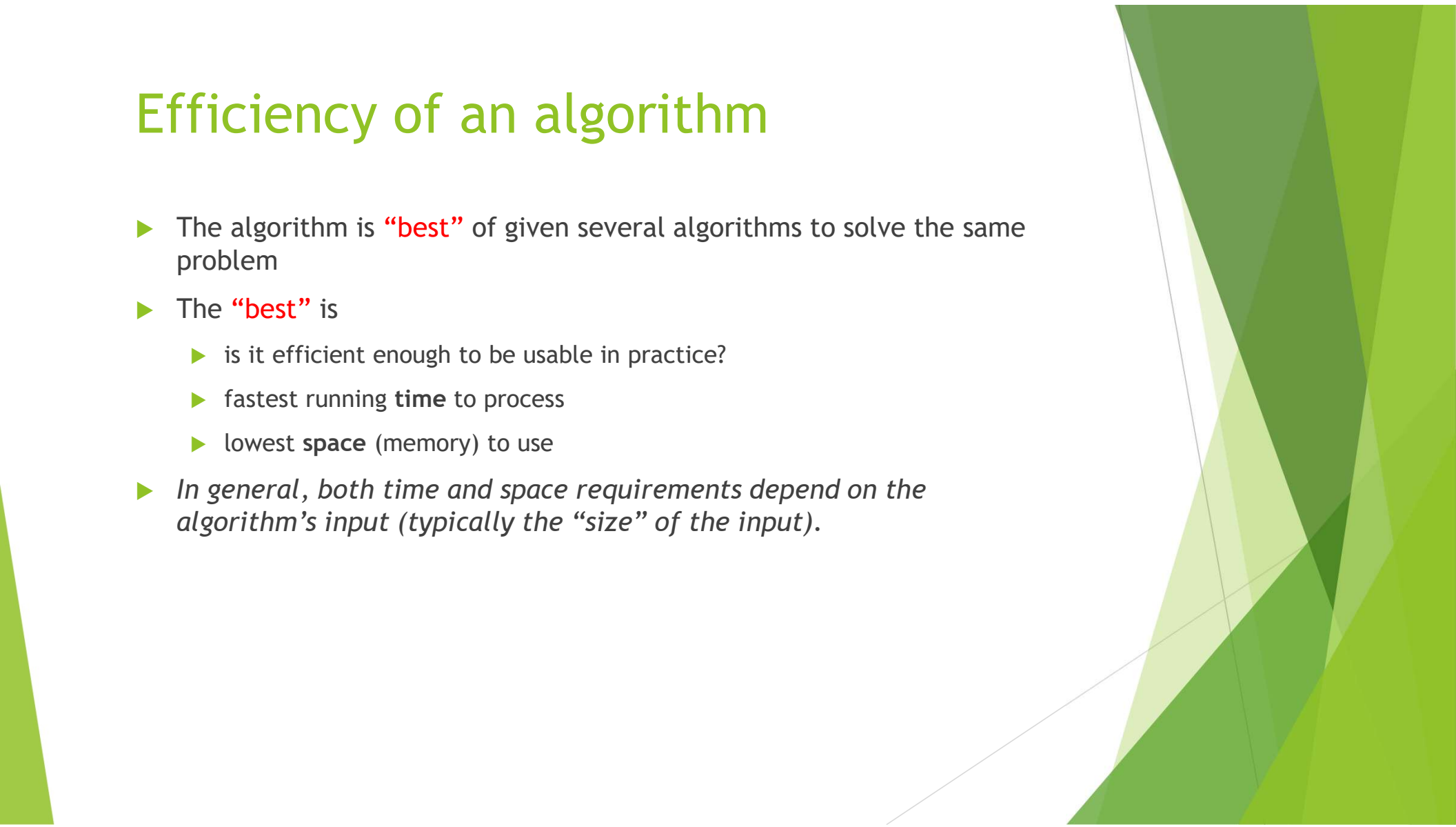


is an algorithm Efficient?



Efficiency of an algorithm

- The algorithm is "best" of given several algorithms to solve the same problem
- The "best" is
  - is it efficient enough to be usable in practice?
  - fastest running **time** to process
  - lowest **space** (memory) to use
- *In general, both time and space requirements depend on the algorithm's input (typically the "size" of the input).*



DBMS algorithms

MySQL

- **MySQL uses B-Tree indexes for efficient retrieval of data.**
- **Efficient querying and indexing of large datasets.**

Efficiency in Practice (80% Popular)

PostgreSQL

- **PostgreSQL uses Hash indexes for equality comparisons.**
- **Fast retrieval of rows based on equality conditions.**

Fastest Running Time

SQLite

- **SQLite is lightweight and uses a compact form of B-Tree indexing to maintain low memory usage.**
- **Embedded databases in mobile applications with limited storage space.**

Lowest Space Usage

Web Browser algorithms

Google Chrome

- **Just-In-Time (JIT) compilation and inline caching to optimize JavaScript execution.**
- **Efficient execution of web applications, leading to faster page loads and smoother interactions.**

Efficiency in Practice (90% Popular)

Mozilla Firefox

- **IonMonkey JIT compiler optimizes JavaScript by compiling frequently executed code into machine code, reducing execution time.**
- **Fast execution of complex JavaScript code in web applications.**

Fastest Running Time

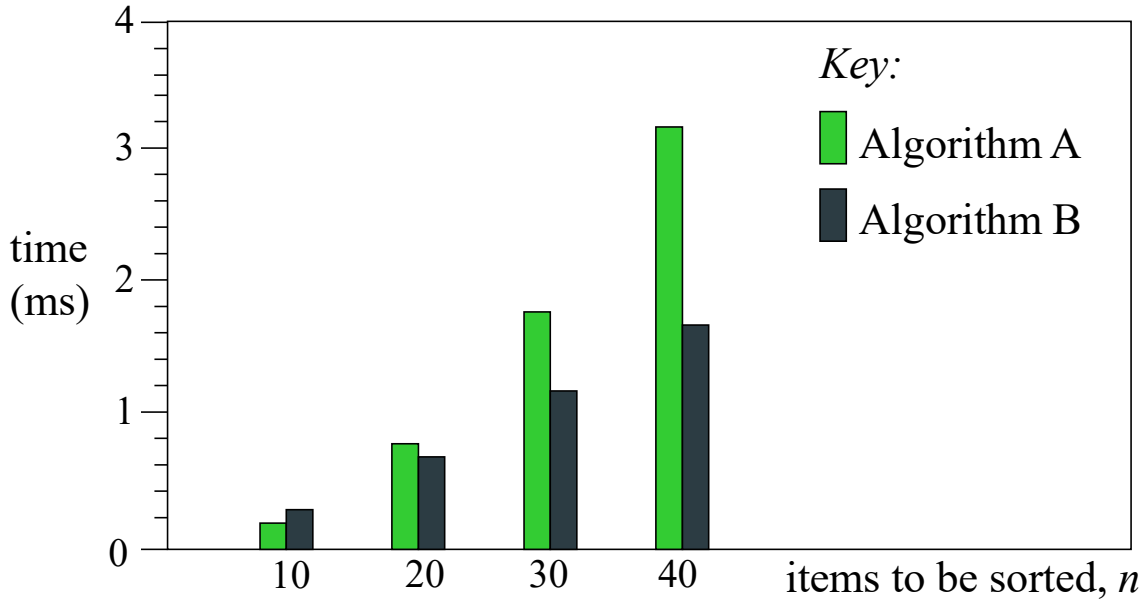
Opera Mini

- **Server-side data compression to minimize the amount of data transmitted to the device, reducing memory and bandwidth usage.**
- **Efficient web browsing on devices with limited storage and slow internet connections.**

Lowest Space Usage

Example: “running time” efficiency

- Hypothetical profile of two sorting algorithms:

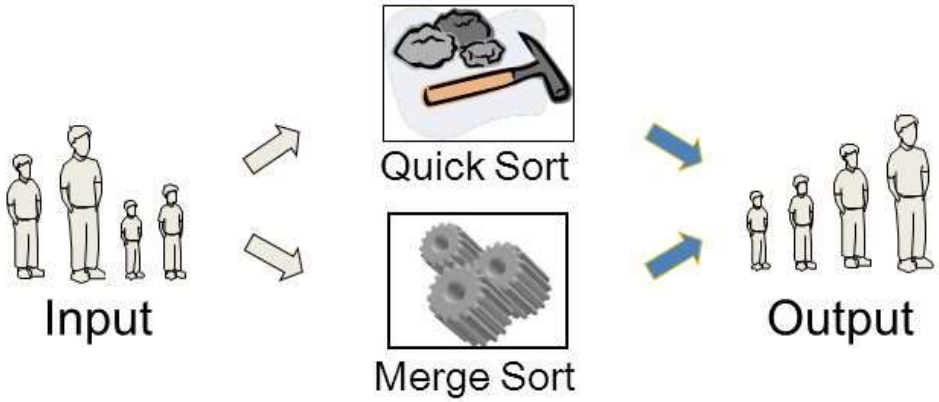


- Algorithm B’s time grows more slowly than A’s.

Analysis of Algorithms

An **algorithm** is a step-by-step procedure for solving a problem in a finite amount of time.

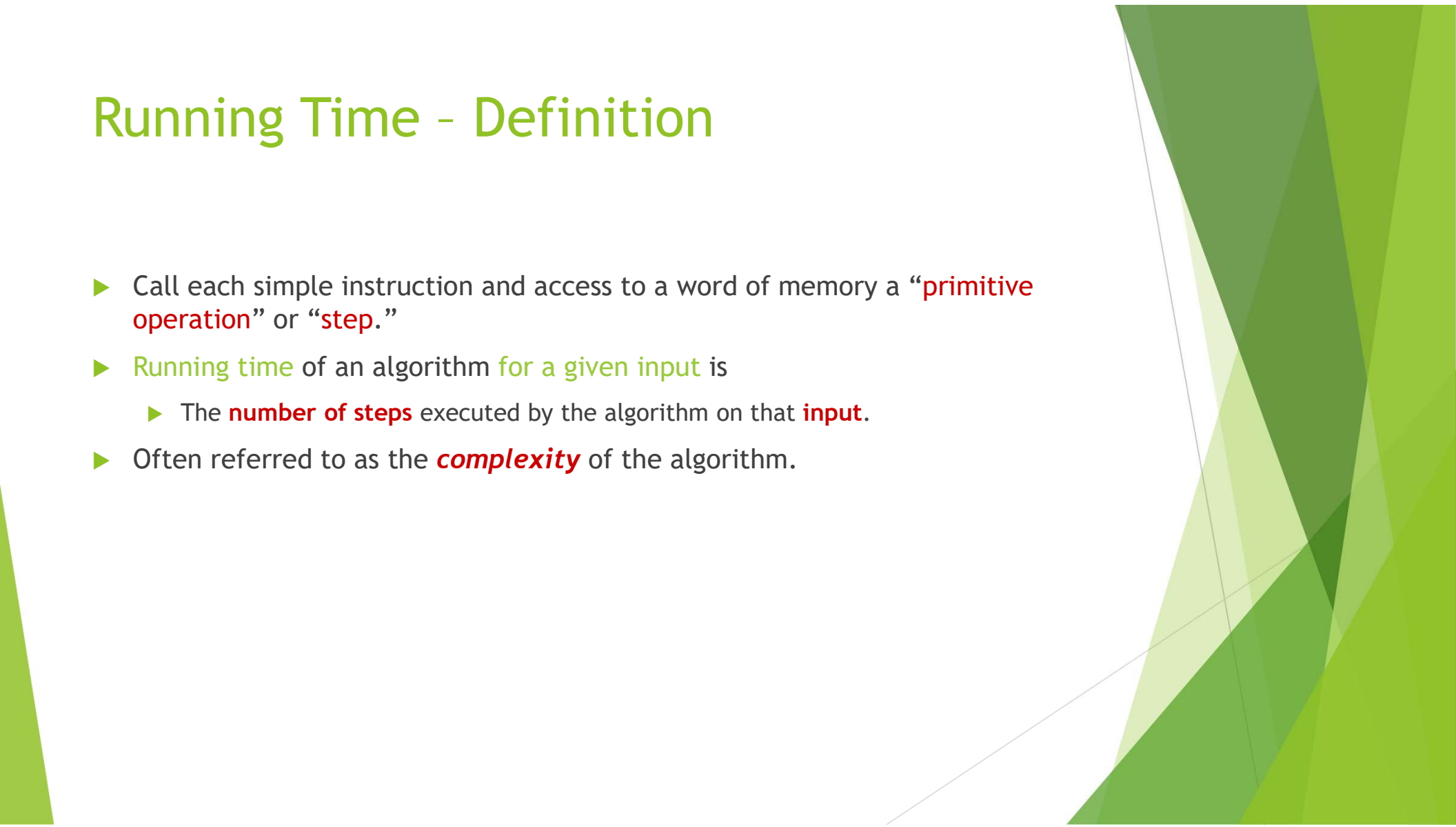
How to evaluate algorithms?



- Which one is better?
- What are the criteria?

Running Time – Definition

- Call each simple instruction and access to a word of memory a "primitive operation" or "step."
- Running time of an algorithm for a given input is
  - The **number of steps** executed by the algorithm on that **input**.
- Often referred to as the ***complexity*** of the algorithm.



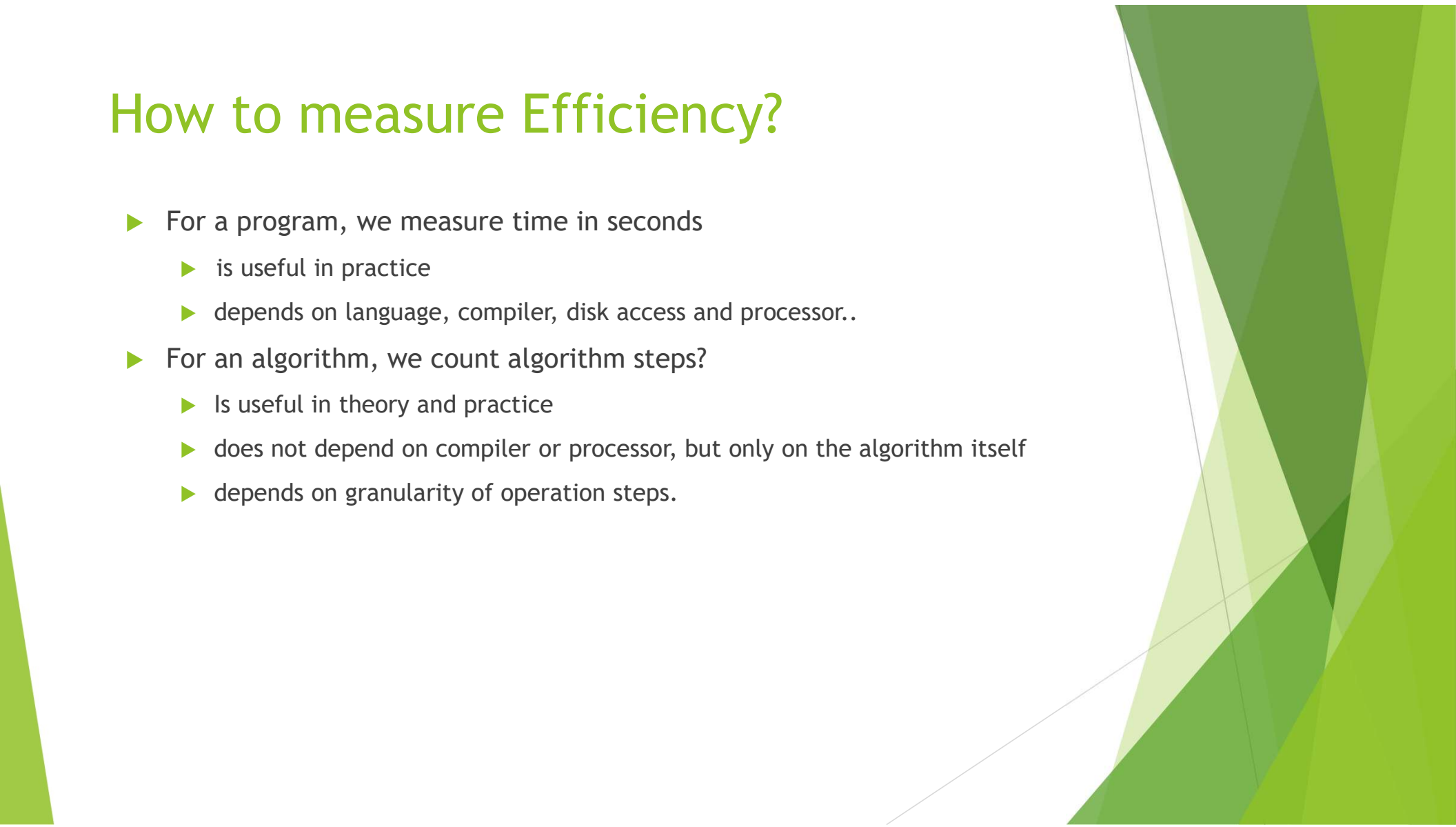
Complexity and Input

- Complexity of an algorithm generally depends on
  - **Size of input**.
    - Input size depends on the problem.
      - Examples: No. of items to be sorted.
      - No. of vertices and edges in a graph.
  - **Other characteristics of the input data**.
    - Are the items already sorted?
    - Are there cycles in the graph?



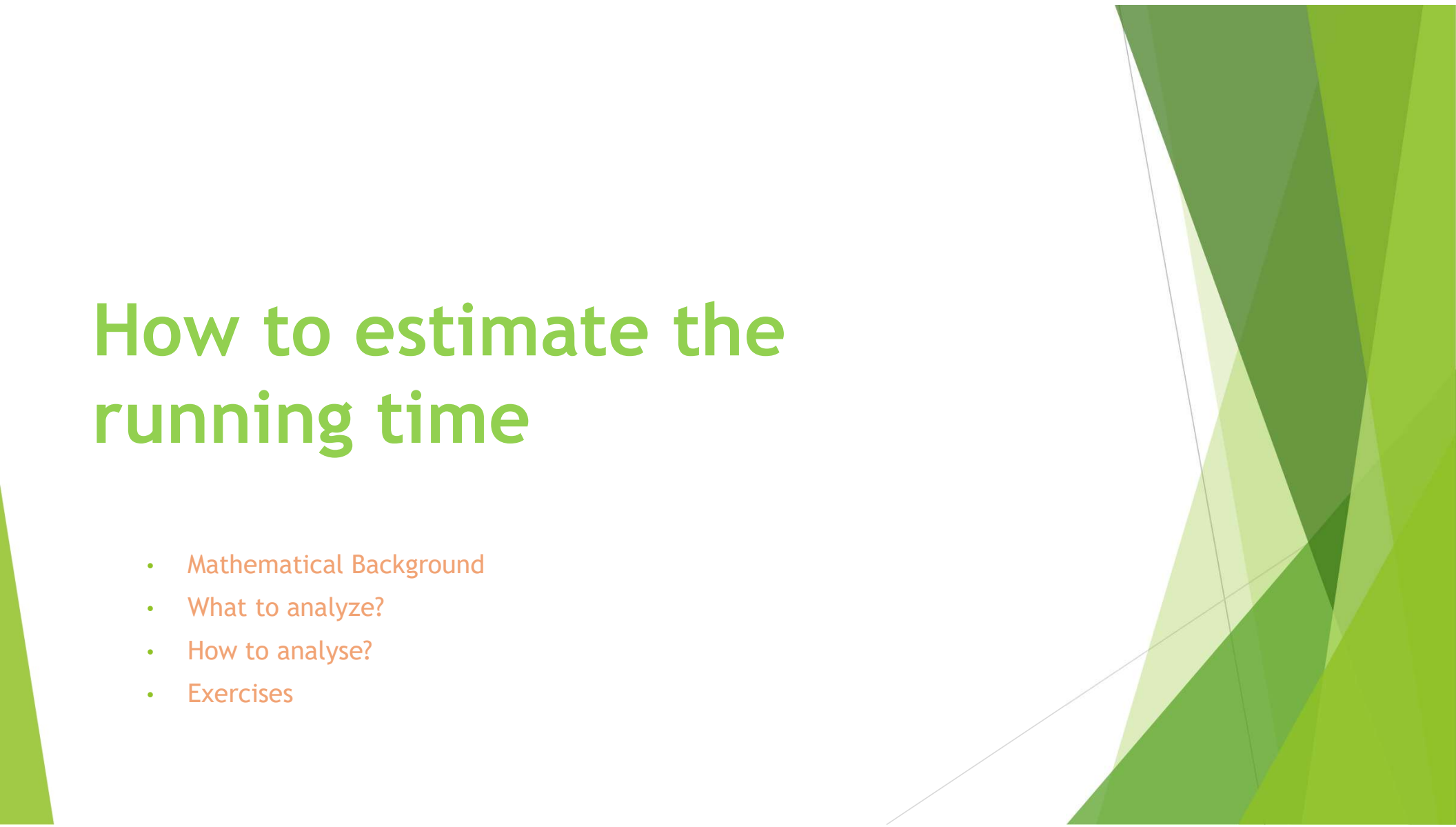
How to measure Efficiency?

- For a program, we measure time in seconds
  - is useful in practice
  - depends on language, compiler, disk access and processor..
- For an algorithm, we count algorithm steps?
  - Is useful in theory and practice
  - does not depend on compiler or processor, but only on the algorithm itself
  - depends on granularity of operation steps.



How to estimate the running time

- Mathematical Background
- What to analyze?
- How to analyse?
- Exercises



Mathematical Background



Powers

- Consider a number b and a non-negative integer n.
  Then ***b* to the power of *n*** (written b^n) is the multiplication of n copies of b:

$$b^n = b \times \ldots \times b$$

E.g.:

$$b^3 = b \times b \times b$$
$$b^2 = b \times b$$
$$b^1 = b$$
$$b^0 = 1$$

Logarithms

- Consider a positive number *y*. Then the **logarithm of *y* to the base 2** (written **$\log_2 y$**) is the number of copies of 2 that must be multiplied together to equal *y*.
- If *y* is a power of 2, $\log_2 y$ is an integer:

E.g.:
$\log_2 1 = 0$ since $2^0 = 1$
$\log_2 2 = 1$ since $2^1 = 2$
$\log_2 4 = 2$ since $2^2 = 4$
$\log_2 8 = 3$ since $2^3 = 8$

- If y is not a power of 2, $\log_2 y$ is fractional:

E.g.: $\log_2 5 \approx 2.32$
$\log_2 7 \approx 2.81$

Logarithm laws

- $\log_2 (2^n) = n$
- $\log_2 (x \times y) = \log_2 x + \log_2 y$
- $\log_2 (x / y) = \log_2 x - \log_2 y$



Logarithms example (1)

- How many times must we halve the value of n (discarding any remainders) to reach 1?
- Suppose that n is a power of 2:

  E.g.: $8 \Rightarrow 4 \Rightarrow 2 \Rightarrow 1$ (8 must be halved 3 times)
  $16 \Rightarrow 8 \Rightarrow 4 \Rightarrow 2 \Rightarrow 1$ (16 must be halved 4 times)

  If $n = 2^m$, n must be halved m times.
- Suppose that n is not a power of 2:

  E.g.: $9 \Rightarrow 4 \Rightarrow 2 \Rightarrow 1$ (9 must be halved 3 times)
  $15 \Rightarrow 7 \Rightarrow 3 \Rightarrow 1$ (15 must be halved 3 times)

  If $2^m < n < 2^{m+1}$, n must be halved m times.

Logarithms example (2)

- In general, n must be halved m times if:

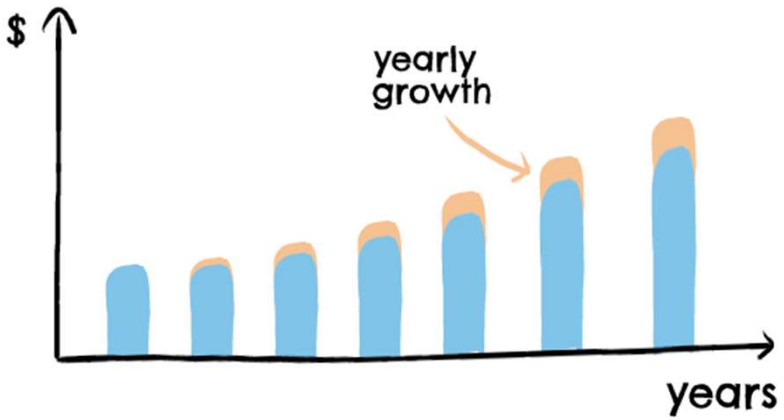
  $2^m \leq n < 2^{m+1}$
  i.e., $\log_2 (2^m) \leq \log_2 n < \log_2 (2^{m+1})$
  i.e., $m \leq \log_2 n < m+1$
  i.e., $m = \text{floor}(\log_2 n)$.

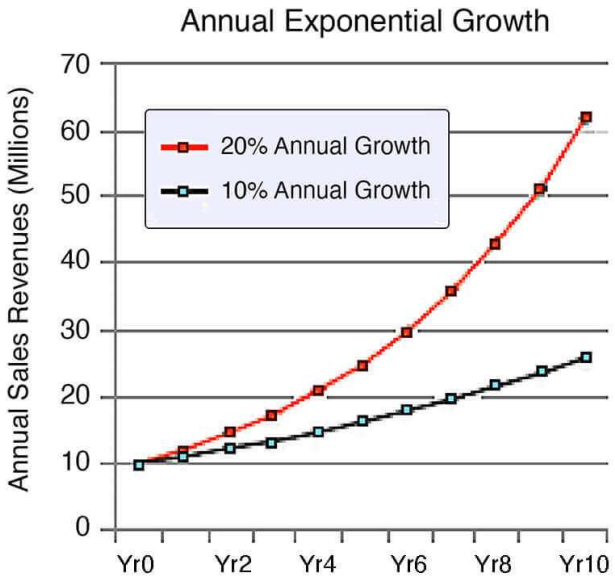
- The **floor of *x*** (written **floor(*x*)** or $\lfloor x \rfloor$) is the largest integer not greater than *x*.
- Conclusion: n must be halved floor($\log_2 n$) times to reach 1.
- Also: n must be halved floor($\log_2 n$)+1 times to reach 0.

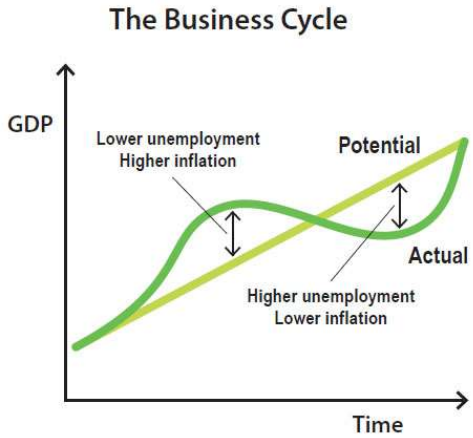
How to compare two functions?

- The idea of the following definitions is to establish a relative order among functions.
- Given two functions, there are usually points where one function is smaller than the other function.
  - It does not make sense to claim $f(N) < g(N)$
  - Let us compare $f(N) = 1000N$ and $g(N) = N^2$
  - Thus, we will compare their relative rates of growth

Growth rates







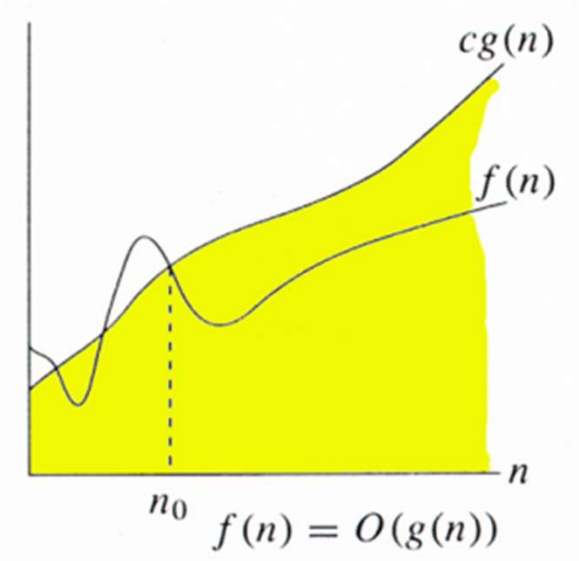
Mathematical background

- **Definition 1:**

  $f(N) = O(g(N))$

  if there are positive constants c and n_0 such that

  $f(N) \le c \cdot g(N)$ when $N \ge n_0$



- **Meaning:**

$f(N) = O(g(N))$ means that

the growth rate of $f(N)$ is less than or equal to that of $g(N)$

Because

$f(N)$ grows at a rate no faster than $g(N)$, Thus $g(N)$ is an upper bound on $f(N)$

Mathematical background

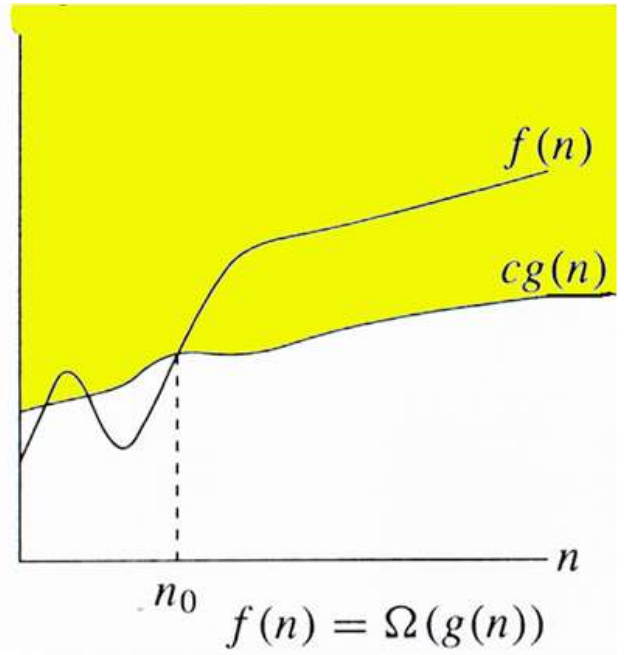
- **Definition 2:**

  $f(N) = \Omega(g(N))$ if there are positive constants c and n_0 such that $f(N) \geq c \cdot g(N)$ when $N \geq n_0$

- **Meaning**

  $f(N) = \Omega(g(N))$ says that the growth rate of $f(N)$ is greater than or equal to that of $g(N)$

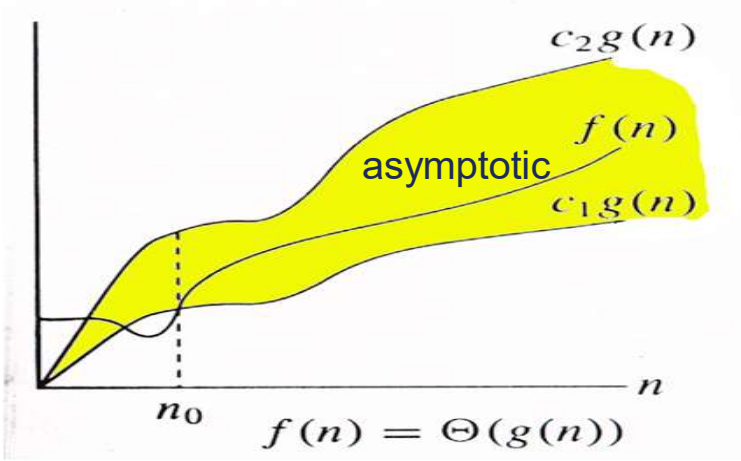
  - $f(N)$ grows at a rate no slower than $g(N)$
    - Thus $g(N)$ is a lower bound on $f(N)$



Mathematical background

- Definition 3:

$f(N) = \Theta(g(N))$ if and only if
$f(N) = O(g(N))$ and $f(N) = \Omega(g(N))$



- Meaning

$f(N) = \Theta(g(N))$ says that the growth rate of $f(N)$ equals to that of $g(N)$

Examples

- $f(N)=N^3$ grows faster than $g(N)=N^2$, so
  $g(N) = O(f(N))$ or $f(N) = \Omega(g(N))$
- $f(N)=N^2$ and $g(N)=2N^2$ grow at the same rate, so
  $f(N) = O(g(N))$ and $f(N) = \Omega(g(N))$
- If $f(N)=2N^2$, $g(N)=O(N^4)$, $g(N)=O(N^3)$, $g(N)=O(N^2)$ are all technically correct, but the last one is the best answer.
- Do not say $T(N)=O(2N^2)$ or $T(N)=O(N^2+N)$. The correct form is $T(N)=O(N^2)$.

Mathematical background

- **Rule 1:** If $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$, then
  (a) $T_1(N) + T_2(N) = O(f(N) + g(N))$
  (b) $T_1(N) \times T_2(N) = O(f(N) \times g(N))$

- **Rule 2:** If $T(N)$ is a polynomial of degree k, then $T(N) = \Theta(N^k)$

- **Rule 3:** $\log^k N = O(N)$ for any constant k.
  This tells us that logarithms grow very slowly.

Examples

- Determine which of $f(N)=N \log N$ and $g(N)=N^{1.5}$ grows faster.

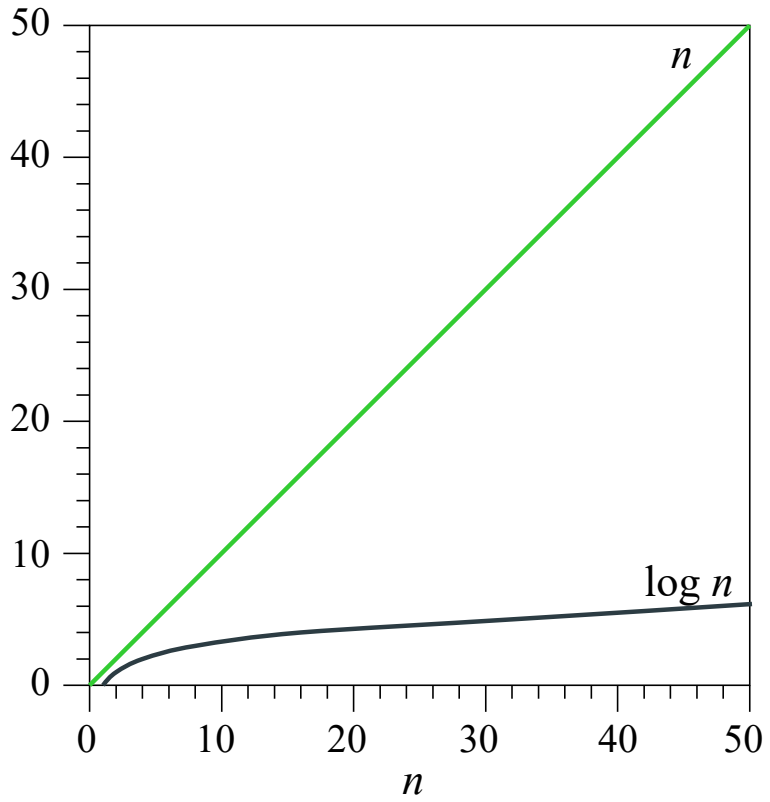
  → Determine which of $\log N$ and $N^{0.5}$ grows faster.

  → Determine which of $\log^2 N$ and N grows faster.

  → Since N grows faster than any power of a log, $g(N)$ grows faster than $f(N)$.

Growing rate of log n - n

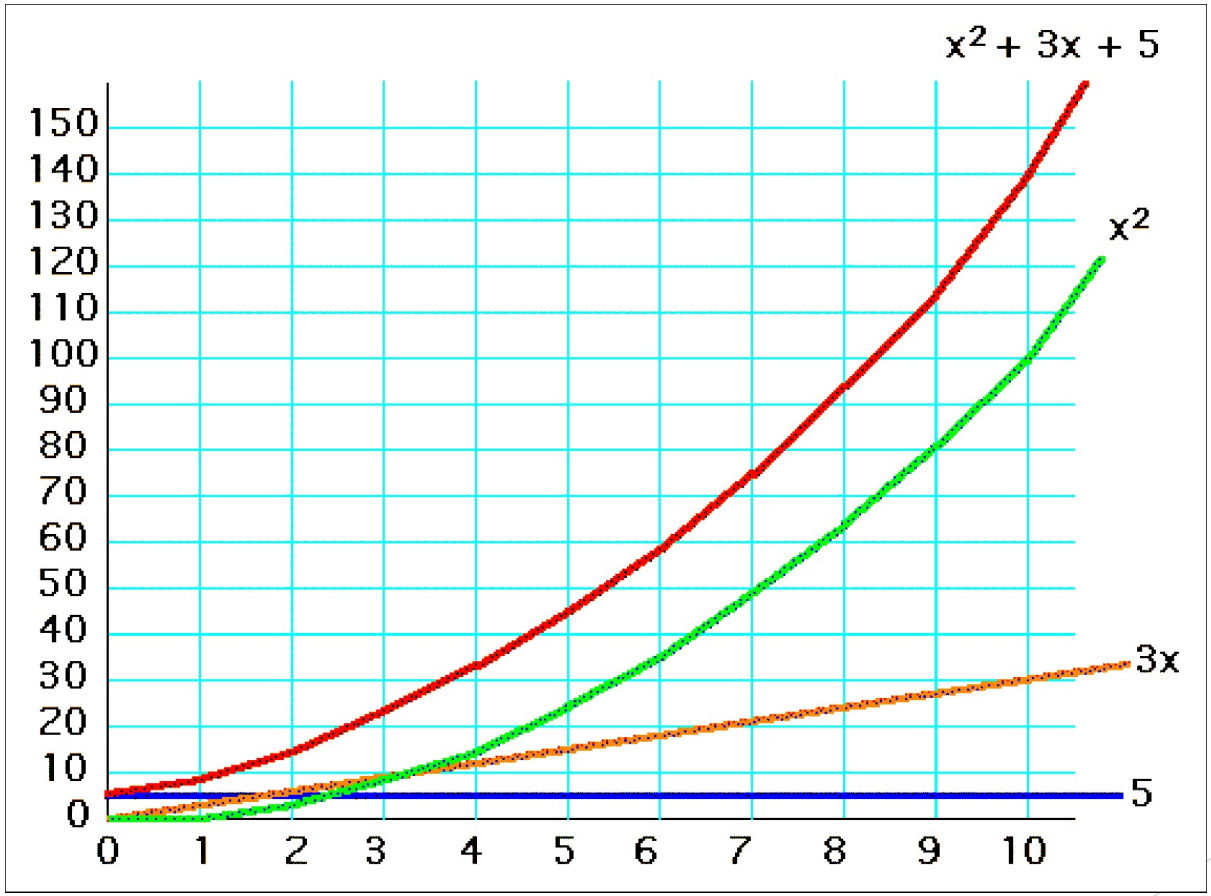
- Comparison:



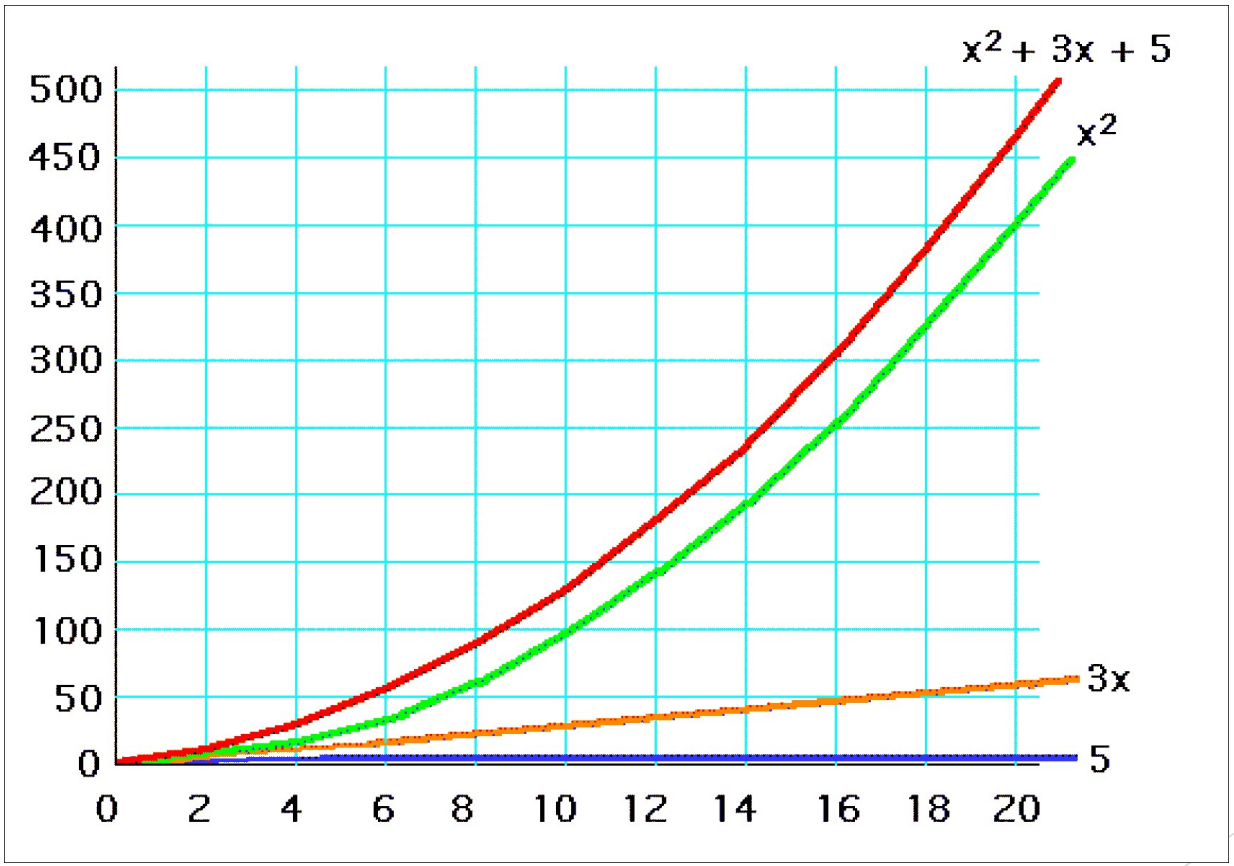
Examples

- Consider the problem of downloading a file over the Internet.
  - Setting up the connection: 3 seconds
  - Download speed: 1.5 Kbytes/second
- If a file is *N* kilobytes, the time to download is $T(N) = N/1.5 + 3$, i.e., $T(N) = O(N)$.
  - 1500K file takes 1003 seconds
  - 750K file takes 503 seconds
- If the connection speed doubles, both times decrease, but downloading 1500K still takes approximately twice the time downloading 750K.

$y = x^2 + 3x + 5$, for x=1..10



$y = x^2 + 3x + 5$, for x=1..20



How to find $O(g(N))$?

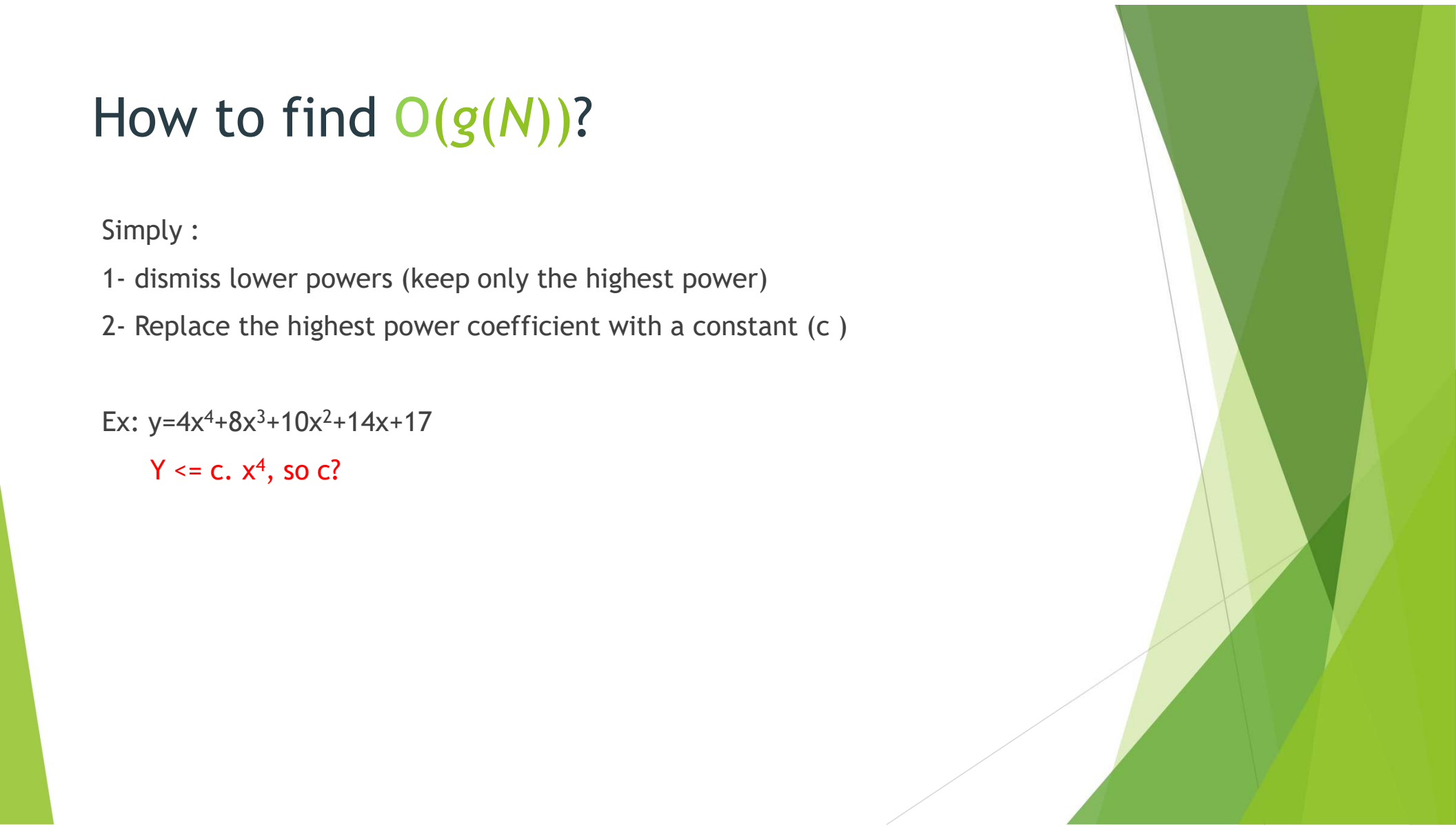
Simply :

1- dismiss lower powers (keep only the highest power)

2- Replace the highest power coefficient with a constant (c)

Ex: $y=4x^4+8x^3+10x^2+14x+17$

$Y <= c.\ x^4$, so c?



What will be the notations?

O (n)	Ω (n)	∂(n)
O (n^4)	Ω (n^4)	∂(n^4)

So what are the difference between them ?
Value of constant and n0

e.x) $f(n) = 4n^4 + 8n^3 + -10n^2 + 14n + 17$
$c.g(n) = c(n^4)$
$f(n) = 4n^4 + 8n^3 + -10n^2 + 14n + 17 \leq CN^4$

ASSUM N0=1 THEN:
$4+8-10+14+17 \leq c$ so $33 \leq c$

More Big-Oh Examples

- 7n-2

  7n-2 is O(n)

  need $c > 0$ and $n_0 \geq 1$ such that $7n-2 \leq c \bullet n$ for $n \geq n_0$

  this is true for $c = 7$ and $n_0 = 1$

- $3n^3 + 20n^2 + 5$

  $3n^3 + 20n^2 + 5$ is $O(n^3)$

  need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq c \bullet n^3$ for $n \geq n_0$

  this is true for $c = 4$ and $n_0 = 21$

- 3 log n + 5

  3 log n + 5 is O(log n)

  need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + 5 \leq c \bullet \log n$ for $n \geq n_0$

  this is true for $c = 8$ and $n_0 = 2$

O-notation

- Common time complexities:

$O(1)$	**constant** time	(feasible)
$O(\log n)$	**logarithmic** time	(feasible)
$O(n)$	**linear** time	(feasible)
$O(n \log n)$	**log linear** time	(feasible)
$O(n^2)$	**quadratic** time	(sometimes feasible)
$O(n^3)$	**cubic** time	(sometimes feasible)
$O(2^n)$	**exponential** time	(rarely feasible)

Common time complexities



- $O(1)$ constant time
- $O(\log n)$ log time
- $O(n)$ linear time
- $O(n \log n)$ log linear time
- $O(n^2)$ quadratic time
- $O(n^3)$ cubic time
- $O(n^k)$ polynomial time
- $O(2^n)$ exponential time

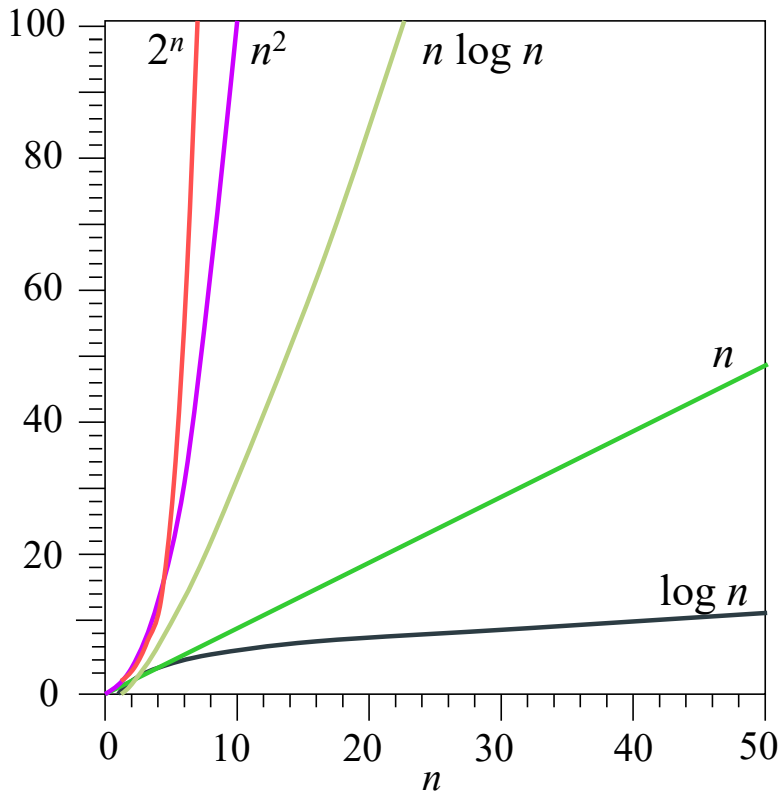
Growth rates

- Comparison of growth rates:

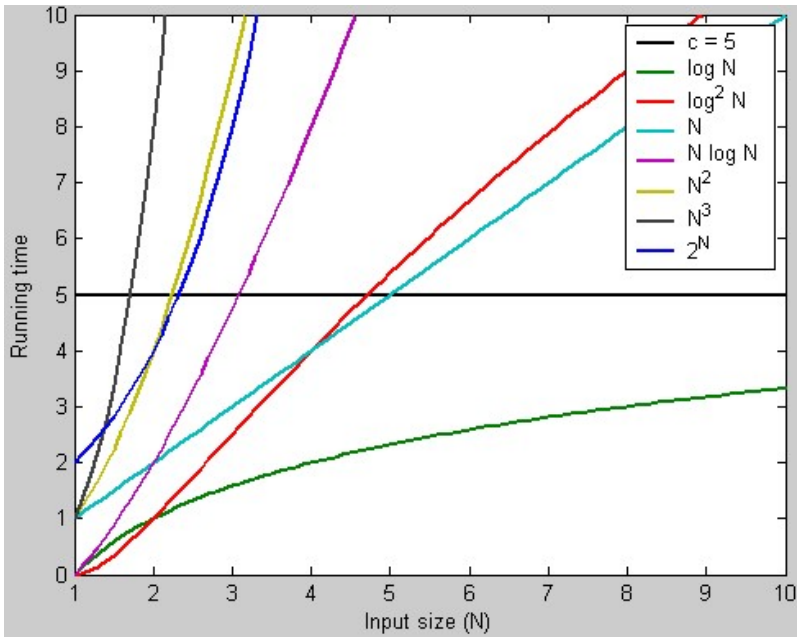
1	1	1	1	1
$\log n$	3.3	4.3	4.9	5.3
n	10	20	30	40
$n \log n$	33	86	147	213
n^2	100	400	900	1,600
n^3	1,000	8,000	27,000	64,000
2^n	1,024	1.0 million	1.1 billion	1.1 trillion

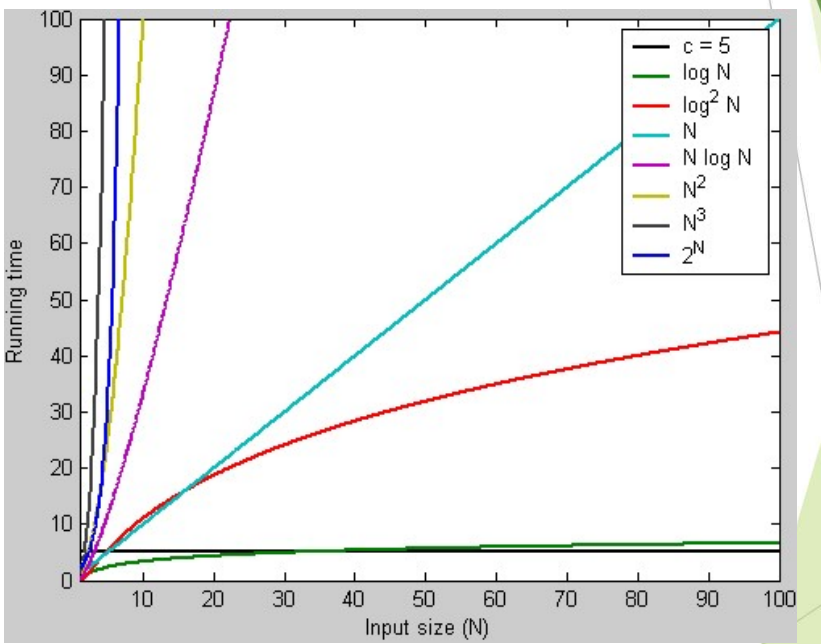
Growth rates (2)

- Graphically:



Plots of various algorithms

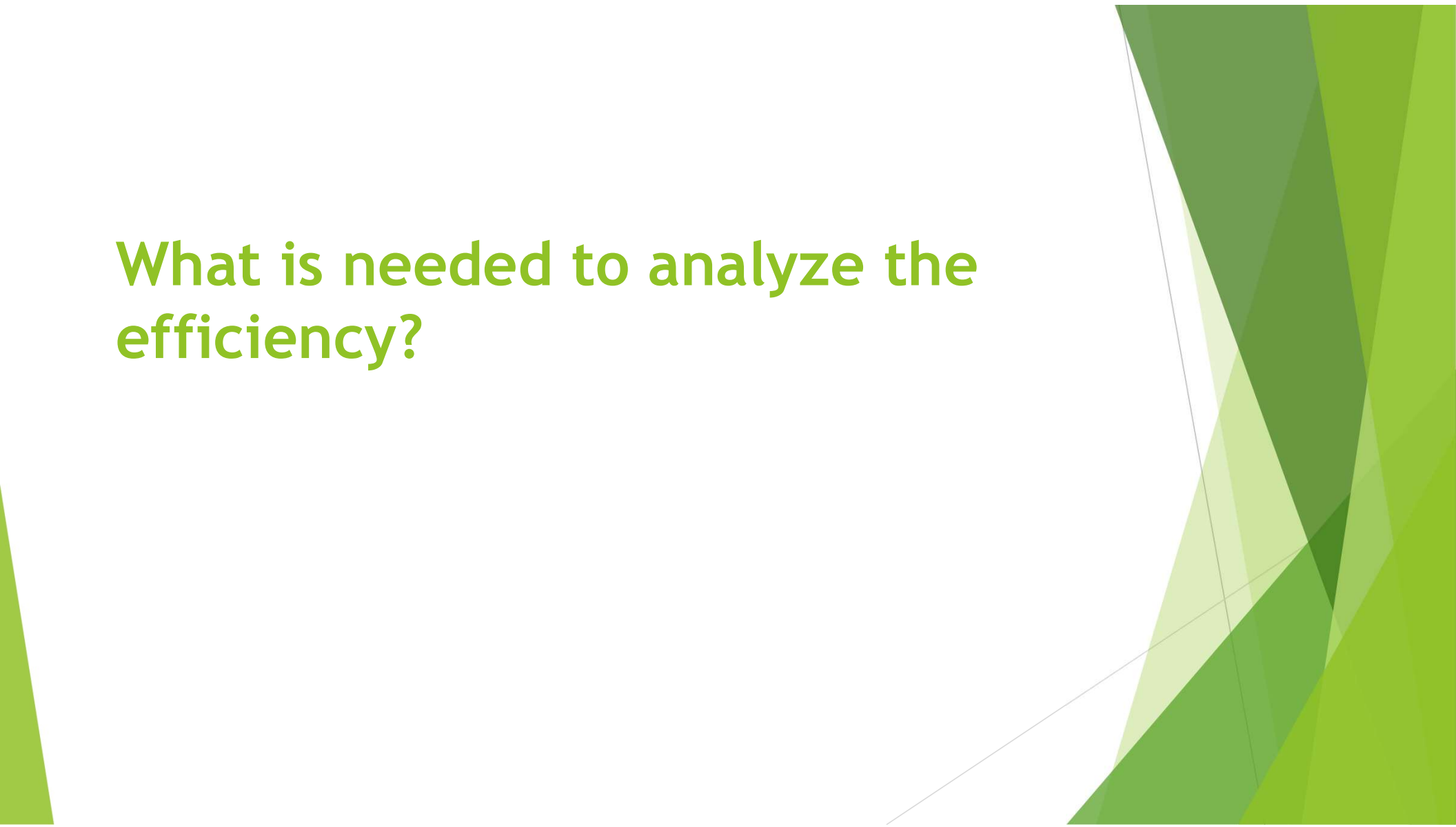




Comparison of Algorithms

- Complexity function can be used to compare the performance of algorithms.
- Algorithm *A* is more efficient than Algorithm *B* for solving a problem, if the complexity function of *A* is of lower order than that of *B*.
- Examples:
  - **Linear Search** – $\Theta(n)$ vs. **Binary Search** – $\Theta(\lg n)$
  - **Insertion Sort** – $\Theta(n^2)$ vs. **Quick Sort** – $\Theta(n \lg n)$

What is needed to analyze the efficiency?



We analyze algorithms, not programs

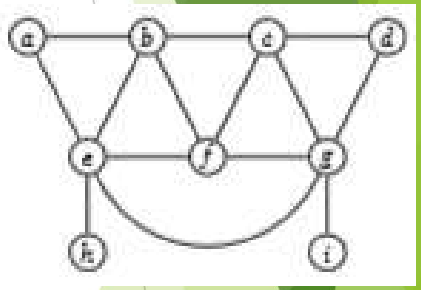
- The most important resource to analyze is generally the Running time
- Identify simple instructions
  - Identify the basic operations
  - Count the number of basic operations
  - Use Big O to describe the upper bound of the algorithm's running time
- Basic operation is not fancy operations such as matrix inversion, list insertion, and sorting
- Do not consider
  - **Practical Constraints:** memory management, input/output operations, and interaction with hardware and operating systems
  - **Real-World Performance:** Compiler optimizations, cache performance, and system architecture
  - **Usability and Maintenance:** user-friendly, maintainable, and scalable

Running Time – Definition

- Simple instruction is “basic operation” or “step”
- Running time is the number of steps of an algorithm to complete its execution as a function of the size of its input
- ***Complexity*** of the algorithm is bound functions of input size.
  - Big O notation is used to describe the upper bound of the running time, focusing on the worst-case scenario

Complexity and Input

- Complexity of an algorithm generally depends on input
  - The amount of time an algorithm takes to complete its execution as a function of the input size
  - Helps predict the algorithm's running time and its scalability
- Input size depends the input data
    - Sorted vs. Unsorted: Some algorithms, like insertion sort, run faster on sorted data compared to unsorted data.
    - Presence of Cycles: Input size can be defined by the number of vertices (nodes) and edges (connections) in the graph
- The nature of the input data also significantly influences the algorithm's complexity



What does “size of the input” mean?

- The "size of the input" refers to a quantitative measure that represents the amount of data the algorithm must process
- Input size can vary depending on the type of problem
  - Number of Elements in a Collection
    - **Example**: If you need to sort an array of 100 integers, the input size n is 100
  - Length of a String
    - **Example**: If you need to find a substring in a string of 256 characters, the input size n is 256
  - Number of Vertices and Edges in a Graph
    - **Example**: If you have a graph with 50 vertices and 200 edges, the input size can be described by V=50 and E=200
  - Number of Records in a Database
  - Dimensions of a Matrix

Comparisons of Algorithms

Sorting

- insertion sort: $\Theta(n^2)$
- merge sort: $\Theta(n \lg n)$

For a sequence of 10^6 numbers,

- the insertion sort took 5.56 hrs on a supercomputer using machine language;
- the merge sort took 16.67 min on a PC using C/C++.

Worst, Average, and Best-case Complexity

Worst-case Complexity

- **Maximum** steps the algorithm takes for any possible input.
- Most tractable measure.

Average-case Complexity

- **Average** of the running times of all ***possible*** **inputs.**
- Demands a definition of probability of each input, which is usually difficult to provide and to analyze.

Best-case Complexity

- **Minimum** number of steps for any possible input.
- Not a useful measure. Why?

Examples (1)

- **Algorithm**: Linear search for finding an element in an array
  - **Input Size**: n (the length of the array)
  - **Running Time**:
  - **Best Case**: O(1)
    - Occurs when the target element is the first element in the array.
  - **Worst Case**: O(n)
    - Occurs when the target element is the last element in the array or not present at all.
  - **Average Case**: O(n)
    - On average, the target element will be found halfway through the array.

Examples (2)

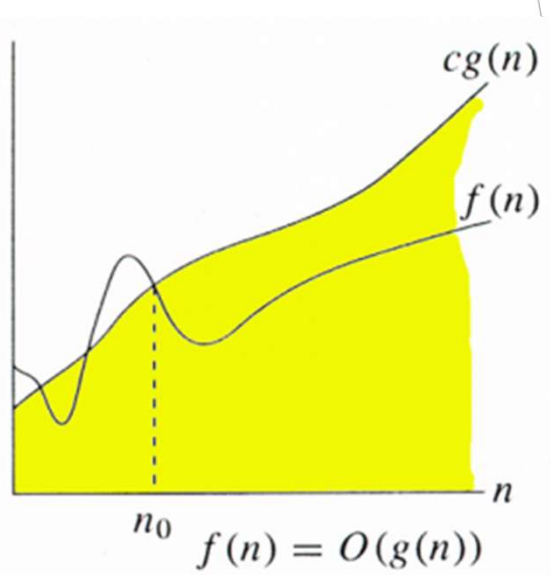
- **Algorithm**: Binary search on a sorted array
  - **Input Size**: n (the length of the sorted array)
  - **Running Time**:
  - **Best Case**: O(1)
    - Occurs when the target element is the middle element of the array.
  - **Worst Case**: O(logn)
    - Occurs when the search space is halved at each step until the target is found or the search space is empty.
  - **Average Case**: O(logn)
    - Generally, the average case running time is similar to the worst case because the search space is repeatedly halved.

Examples (3)

- **Algorithm**: QuickSort for sorting an array
  - **Input Size**: n (the length of the array)
  - **Running Time**:
  - **Best Case**: O(nlogn)
    - Occurs when the pivot element always divides the array into two equal halves.
  - **Worst Case**: $O(n^2)$
    - Occurs when the pivot element is always the smallest or largest element, resulting in the most unbalanced partitions.
  - **Average Case**: O(nlogn)
    - Typically, the average case running time is close to the best case due to the expected balanced partitioning.

Big-O typically used in Algorithm analysis

- Function of growth rate shows detailed information of an algorithm's performance, Big O offers several advantages
  - **Simplification:** focusing on the dominant growth rate
  - **Generalization:** Provides a general upper bound and captures the worst-case scenario, offering a broad perspective on performance
  - **Comparability:** facilitating easy comparison and understanding



Algorithm analysis assumption

- Each basic operation (such as addition, comparison, and assignment) takes a constant amount of time
- The system has infinite memory available, so memory allocation and usage do not impose constraints on the algorithm
- The input size is large enough that only the dominant terms and higher-order growth rates matter
- he analysis often focuses on the worst-case scenario, where the input causes the algorithm to take the longest time to complete
- The algorithm proceeds in discrete steps, with each step corresponding to a basic operation
- The algorithm runs on a single processor, and parallel or distributed computing aspects are not considered

Thus, Algorithm analysis is to eliminate the bad algorithmic ideas early before coding

Examples…



A Simple Example – *Linear Search*

INPUT: a sequence of *n* numbers, *key* to search for.

OUTPUT: *true* if *key* occurs in the sequence, *false* otherwise.

LinearSearch(A, *key*)	*cost*	*times*
1 $i \leftarrow 1$	c_1	1
2 **while** $i \leq n$ **and** A[i] != *key*	c_2	x
3 **do** i++	c_3	x-1
4 **if** $i \leq n$	c_4	1
5 **then return** *true*	c_5	1
6 **else return** *false*	c_6	1

x ranges between 1 and n+1.

So, the running time ranges between

$c_1 + c_2 + c_4 + c_5$ – **best case**

and

$c_1 + c_2(n+1) + c_3 n + c_4 + c_6$ – **worst case**

A Simple Example – *Linear Search*

INPUT: a sequence of *n* numbers, *key* to search for.

OUTPUT: *true* if *key* occurs in the sequence, *false* otherwise.

LinearSearch(A, *key*)	*cost*	*times*
1 $i \leftarrow 1$	1	1
2 **while** $i \leq n$ **and** A[i] != *key*	1	x
3 **do** i++	1	x-1
4 **if** $i \leq n$	1	1
5 **then return** *true*	1	1
6 **else return** *false*	1	1

Assign a cost of 1 to all statement executions.

Now, the running time ranges between

$1+1+1+1=4$ – **best case**

and

$1+(n+1)+n+1+1=2n+4$ – **worst case**

A Simple Example – *Linear Search*

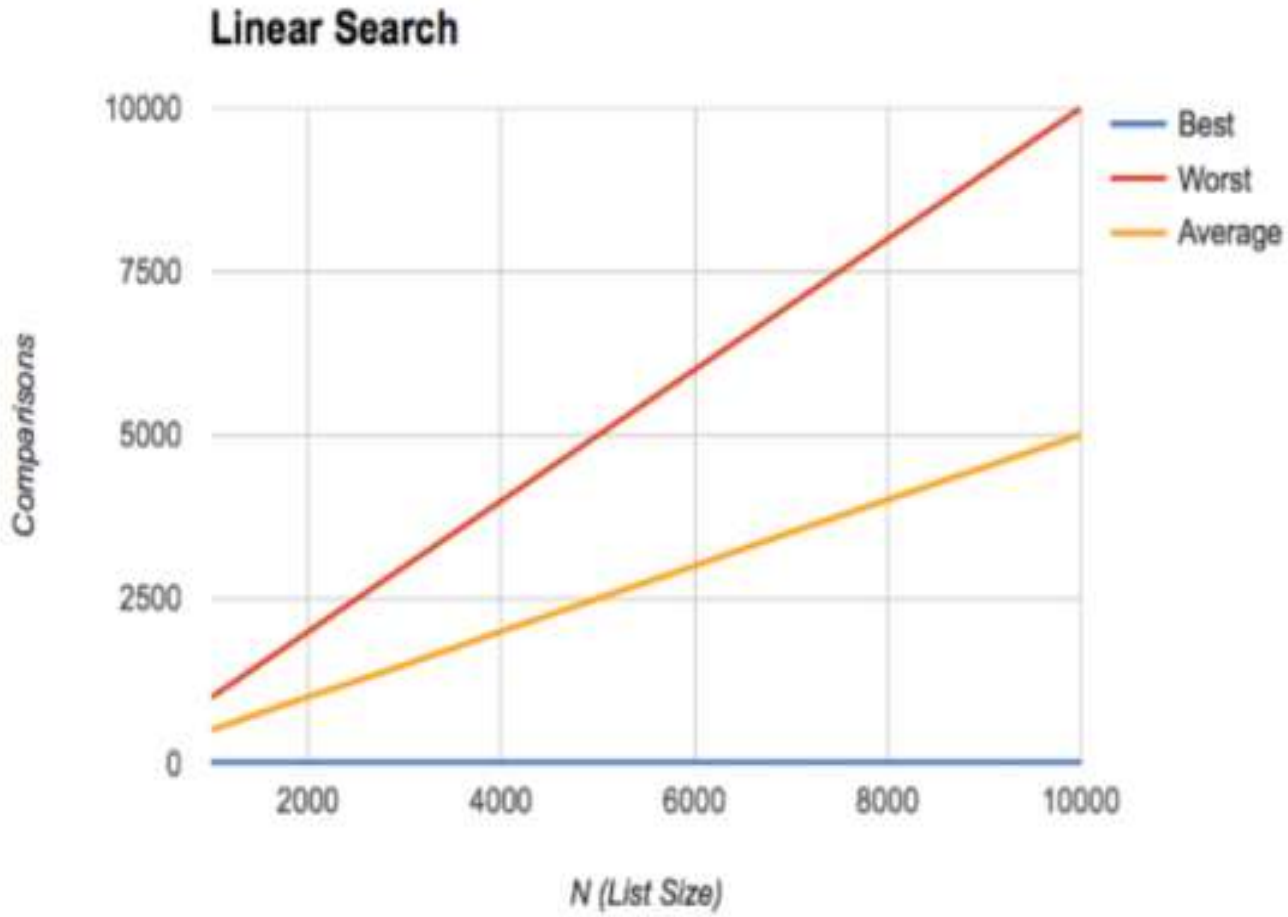
INPUT: a sequence of *n* numbers, *key* to search for.

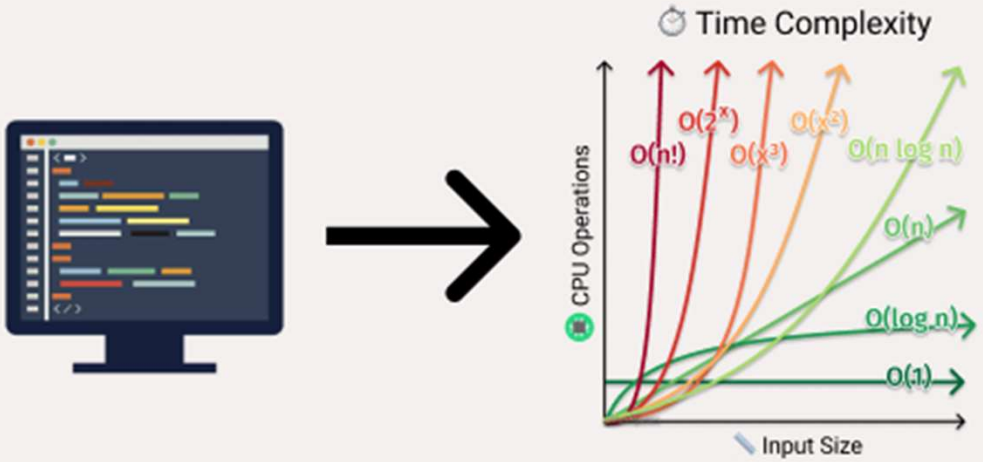
OUTPUT: *true* if *key* occurs in the sequence, *false* otherwise.

LinearSearch(A, *key*)	*cost*	*times*
1 $i \leftarrow 1$	1	1
2 **while** $i \leq n$ **and** A[i] != *key*	1	x
3 **do** i++	1	x-1
4 **if** $i \leq n$	1	1
5 **then return** *true*	1	1
6 **else return** *false*	1	1

If we assume that we search for a random item in the list, on average, Statements 2 and 3 will be executed $n/2$ times. Running times of other statements are independent of input. Hence, **average-case complexity** is

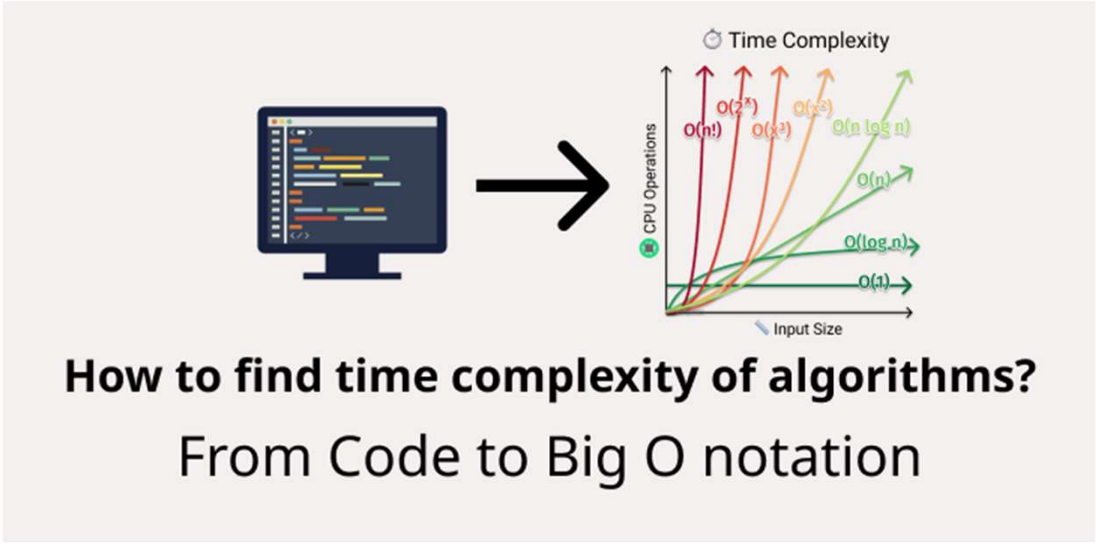
$1+ n/2+ n/2 + 1 + 1 = n+3$


Linear Search
10000
7500
5000
2500
0
Comparisons
2000
4000
6000
8000
10000
N (List Size)
Best
Worst
Average



How to find time complexity of algorithms?

From Code to Big O notation



How to analyse algorithmes

Running Time

- Number of primitive steps that are executed
- Except for time of executing a function call most statements roughly require the same amount of time

Method	Cost
A=1	1
B=A+1	3
C=A*B	4
D=A+B+C	6

EXAMPLE

Method	Cost
A=1	1
B=A+1	3
C=A*B	4
D=A+B+C	6

Method	Cost
A=0	1
FOR I =1 to 3 A= A + i	3

Constant time

- Constant time complexity, O(1), is the running **time** that **remains fixed** regardless of the input size
  - **Fixed Execution Time:** A fixed number of operations that do not depend on the size of the input
- An operation does not include a loop (or call function)
- Algorithms with constant time complexity are the most efficient
  - providing the fastest and most predictable performance



Linear time

- Linear time complexity, O(n), is the running time that **grows linearly** with the size of the input
  - **Proportional Growth**: The running time increases proportionally with the input size

```
for (i = 0, j = 1; i < n; i++) {
    j = j * i;
}
```

  - This loop takes time k*n + c, for some constants k and c

    k : How long it takes to go through the loop once
    *(the time for j = j * i, plus loop overhead)*

    n : The number of times through the loop
    (normally as the "size" of the problem)

    c : The time it takes to initialize the loop
  - The total time k*n + c is *linear in n*



While not as fast as constant time, linear time is efficient and manageable for most practical purposes

Constant time is (usually) better than linear time

- Suppose we have two algorithms to solve a task:
  - Algorithm A takes 5000 time units
  - Algorithm B takes 100*n time units
- Which is better?
  - Clearly, algorithm B is better if input size is small, $n < 50$
  - Algorithm A is better for larger input size, $n > 50$
- We usually care most about very large problems
- But not always!
  - Constant time operations may require additional space
  - Small datasets or infrequent operations

Analyze a simple algorithm

Here is a simple program fragment to calculate $\sum_{i=1}^{N} i^3$:

```
int sum( int n )
{
        int partialSum;


1       partialSum = 0;
2       for( int i = 1; i <= n; ++i )
3           partialSum += i * i * i;
4       return partialSum;
}
```

Count:

- Lines 1 and 4 = 1 + 1 = 2
- Line 3 = 4 x n
- Line 2 = 1 + (n + 1) + n = 2n + 2

Total = 6n + 4

If we had to perform all this work to analyze any algorithm in terms of Big-Oh

- line 3 is obviously *O*(n) time
- Line 1, 4 is constant O(1), so it is silly to waste time here

Impact of Constants in Algorithm Analysis

- **Added Constants:** f(n)+c
  - As the input size n grows, the constant c becomes insignificant.
  - Added constants are less important for large n.
- **Constant Multipliers:** k × f(n)
  - While a constant multiplier k affects the running time, k only provides a linear speedup.
  - **Linear Speedup**: Reducing k by half, the running time is reduced in half
- **Conclusion**
  - Focus on Growth Rates with large inputs
  - Forget the Constants because less relevant to the input size
  - Focus on Complexity is the growth rate for large inputs

Loop rules (1)

- Rule 1 – for loops: The running time of a for loop

Total Running Time = Running Time of operations Inside the Loop × Number of Iterations

  - Algorithm:

    for i from 1 to n

    do constant-time operation

  - Running time: $O(1) \times n = O(n)$

- Rule 2 – nested loops: analyze from the innermost loop outward,

Total Running Time = Running Time of operations × (Product of the Sizes of All Nested Loops)

  - Algorithm:

    for i from 1 to n:

    for j from 1 to m:

    do constant-time operation

  - *Running time:* $O(1) \times m \times n = O(n \times m)$

Example running time for loop

For loop	Running time
for (i = 0; i < n; i++)	n
for (i = 0; i < n; i++) for j = 0; j < n; j++)	n*n = n^2
Int I = 0 Int sum = 0 For 0 to 50	1 1 51

Analyzing the running time (1)

- Nested loops

```
for i = 1 to n do
    for j = 1 to n do
     sum = sum + 1
```

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{n} 1 = \sum_{i=1}^{n} n = n^2$$

Analyzing the running time (2)

- Nested loops

```
for i = 1 to n do
    for j = i to n do
     sum = sum + 1
```

$$T(n) = \sum_{i=1}^{n} \sum_{j=i}^{n} 1 = \sum_{i=1}^{n} (n - i + 1) = \sum_{i=1}^{n} (n+1) - \sum_{i=1}^{n} i$$

$$= n(n+1) - \frac{n(n+1)}{2} = \frac{n(n+1)}{2} = \Theta(n^2)$$

Exercise

- A more complicated example

```
for ( i=0; i<=n; i++)
   for ( j=i; j>=1; j=⌊ j/5⌋ )
    sum = sum + 1
```

T(n) = ?

Loop rules (2)

- Rule 3 – consecutive statements: just add.

```
for (i = 0; i < n; i++)
    a[i] = 0;
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        a[i] += a[j] + i + j;
```

Running time: $n + n \times n = O(n^2)$

- Rule 4 – if/else: the running time is never more than that of the condition plus the larger of that of S1 and S2

```
if (condition)
    S1
else
    S2
```

time ≤ time(C) + Max(time(S1), time(S2))

Example

- *Alg.:* MIN (a[1], …, a[n])

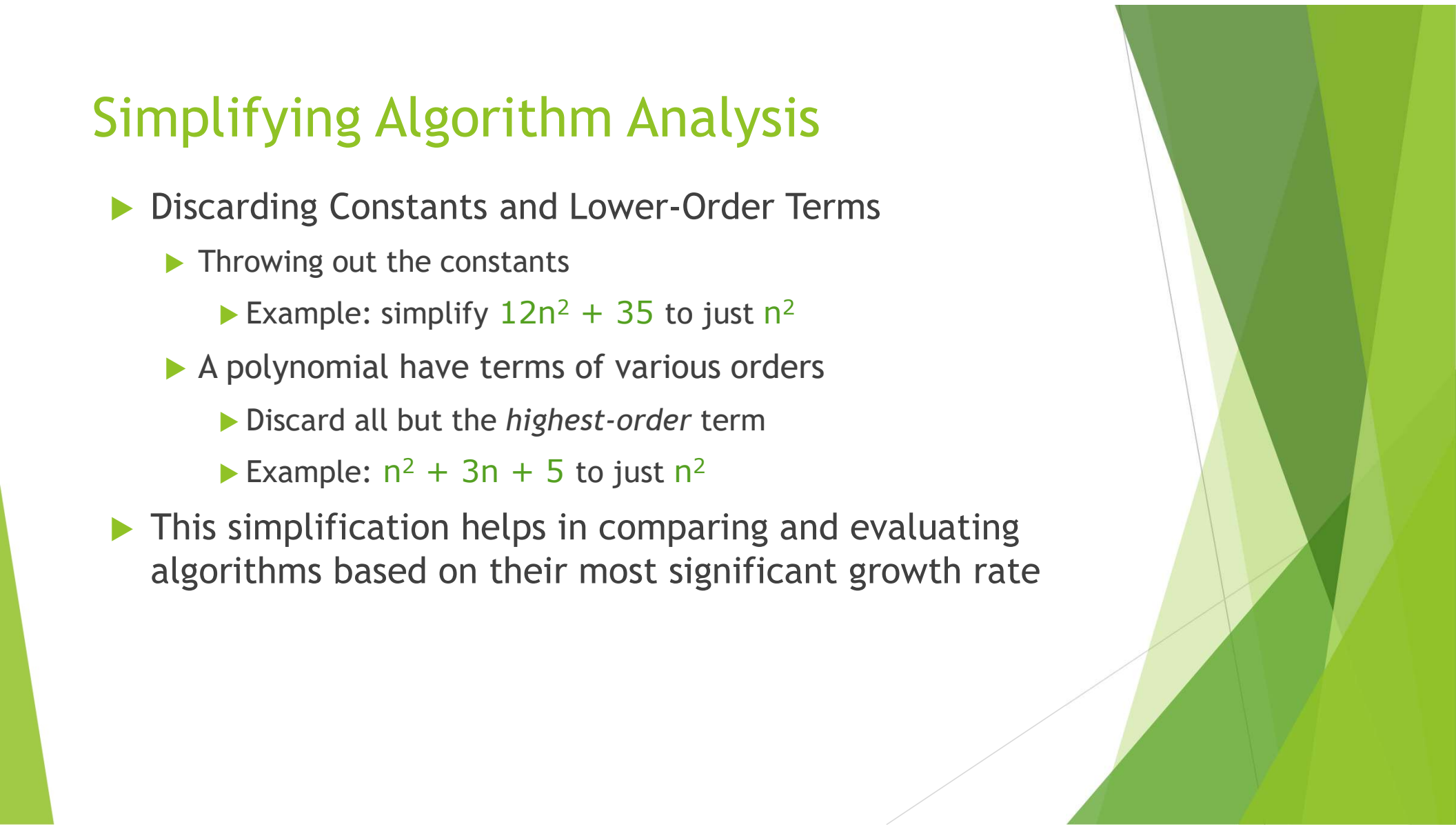
```
m ← a[1];
for i ← 2 to n
    if a[i] < m
    then m ← a[i];
```

- Running time:

$T(n) = 1$ [first step] + (n) [for loop] + $(n-1)$ [if condition] + $(n-1)$ [the assignment in then] = $3n - 1$

Simplifying Algorithm Analysis

- Discarding Constants and Lower-Order Terms
  - Throwing out the constants
    - Example: simplify $12n^2 + 35$ to just n^2
  - A polynomial have terms of various orders
    - Discard all but the *highest-order* term
    - Example: $n^2 + 3n + 5$ to just n^2
- This simplification helps in comparing and evaluating algorithms based on their most significant growth rate



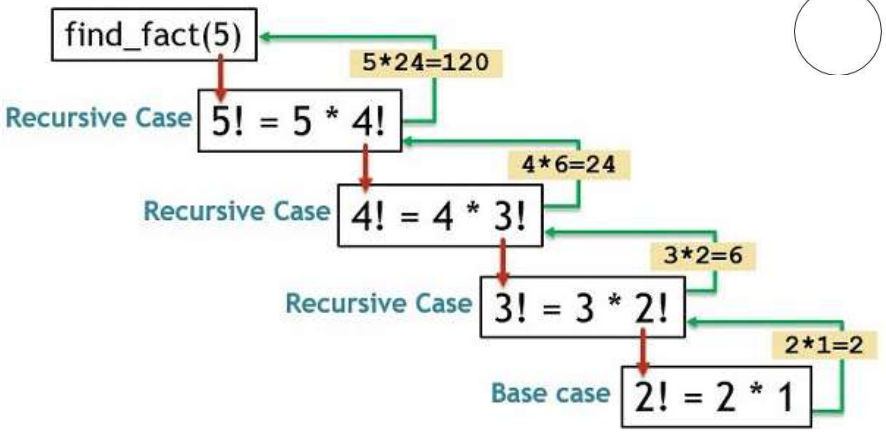
Why using Big O instead of growth rate function?

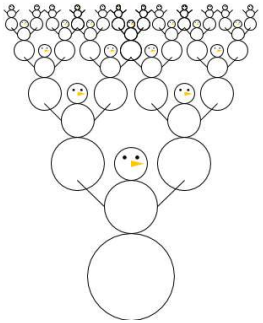
- **Simplification and Abstraction:** This simplification allows us to compare and classify algorithms more easily
- **Generalization:** It captures the worst-case scenario and provides a broad perspective on how the algorithm's performance scales with increasing input sizes
- **Flexibility and Comparability:** Algorithms can be classified into common complexity classes (e.g., O(1), O(log n), O(n), O(n log n), O(n^2), etc.)
- **Focus on Relevant Information:** Focuses on the essential factors that impact efficiency, such as the input size, rather than the specific details of the growth function
- **Emphasis on Worst-case Analysis:** Provides an upper bound on the algorithm's performance across all possible inputs

Recursive functions

- If there are function calls, these must be analyzed first
- If there are recursive functions, be careful about their analyses. For some recursions, the analysis is trivial

```
long factorial(int n){
 if (n <= 1)
     return 1;
  return n * factorial(n-1);
}
```





Recursive running time

- $T(n) = T(n - 1) + 3$
  $T(0) = 1$
- $T(n) = T(n-1) + 3$
  $= T(n-2) + 6$
  $= T(n-3) + 9$
  $= T(n-4) + 12$
  $= \ldots$
  $= T(n-k) + 3k$
- as we know $T(0) = 1$
  we need to find the value of k for which $n - k = 0$, $k = n$
- $T(n) = T(0) + 3n$, $k = n$
  $= 1 + 3n$
- that gives us a time complexity of $O(n)$

Recursive functions

- Let us analyze the following recursion

```
long fib(int n){
    if (n <= 1)
        return 1;
    return fib(n-1) + fib(n-2);
}
```

- $T(N) = T(N-1) + T(N-2) + 2$
- By induction, it is possible to show that $fib(N) < (5/3)^N$ and $fib(N) \geq (3/2)^N$
- *So the running time grows exponentially. By using a for loop, the running time can be reduced substantially*